

MACHINE LEARNING IN FINANCE

MODULE 2 | LESSON 1

UNSUPERVISED MACHINE LEARNING: CLUSTERING

Reading Time	65 minutes
Prior Knowledge	Linear Algebra, machine learning
Keywords	Clustering, K-means, elbow plot

In this lesson, we will learn the difference between supervised learning and unsupervised learning and their applications. Once we have a good understanding of unsupervised learning applications, we will study its foundational techniques. At the end of the lesson, we will implement the k-means clustering algorithm using built-in Python packages.

1. Introduction

In supervised learning, the output (target variable) that we want to predict is known. That allows us to assess the quality of our model by applying metrics such as recall, accuracy, mean squared error, etc., which are used to evaluate the performance of the model. In unsupervised learning, the target variable is not available. The unsupervised model helps in finding patterns in our clustered data without any associated response.

The table below lists some of the differences between unsupervised and supervised models.

Tables:

Unsupervised	Supervised
No labels provided	Labels are provided
Find or explore patterns in unlabeled data	Find or explore patterns in existing structure
Uses techniques such as clustering and dimensionality reduction	Uses techniques such as regression and classification

A supervised classification algorithm can be used to categorize given data into two known groups while an unsupervised clustering algorithm will tell us how the data is structured.

Unsupervised machine learning is applied under the following scenarios

- When the data has no output/target variable.
- When we want to understand patterns in the data.
- To pick out essential information (a process called dimensionality reduction) from the data and use it to train a supervised model.

There are a variety of applications of unsupervised machine learning in the financial industry. Here, we list just a few:

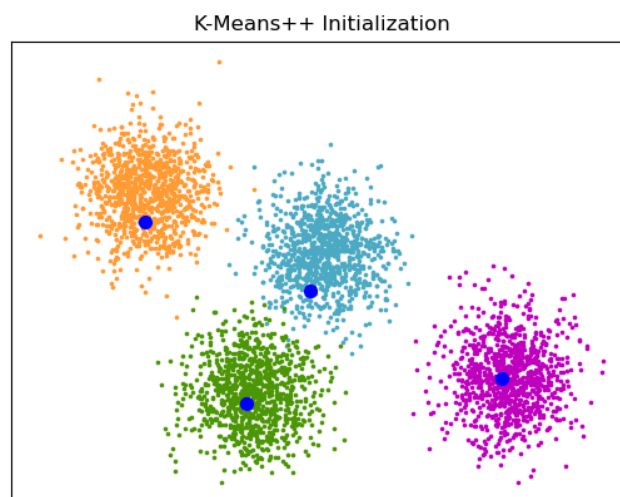
- Asset allocation
- Yield curve construction and interest rate modeling
- Trading to enhance speed and accuracy
- Pairs trading
- Portfolio management to cluster investors

In the rest of this lesson, we will study a branch of unsupervised machine learning called **clustering**.

2. Clustering

Clustering is a process of organizing data points into groups such that members of each group share some similar attributes. In the plot below, we can say that elements in Cluster 0 are similar to each other but different from elements in another cluster, say Cluster 1.

Fig 1: A Plot with Four Clusters



Source: [scikit-learn](#)

As seen in the above plot, we can cluster (group) the data points by using the distance metrics between adjacent datasets.

2.1 Distance-Based Clustering

Given a set of data points, we can use the distance between the points to group the points into clusters such that

- Internal distances between the individual points (intra-cluster) should be small, that is, the points of a given cluster should be close to each other.
- External distances between the points (inter-cluster) should be large, that is, the points in different clusters are dissimilar.

2.2 The Goals of Clustering

Given a set of unlabeled data, we will apply clustering to find the structure and groupings that exist in the data. There exists no perfect criterion for doing clustering, and it is upon the user to supply a criterion that will ensure the result matches their needs.

For any clustering method we opt to use, we need to define an appropriate **similarity measure** to be applied in finding the clusters.

2.3 Proximity Measures

When we do clustering, a proximity measure between two points must be defined where proximity is the data similarity or dissimilarity with respect to each other. The similarity measure $S(x_i, x_k)$ is large if the points x_i, x_k are similar while the dissimilarity measure $D(x_i, x_k)$ is small if the points x_i, x_k are similar.

Below are the similarity measures that we can use:

1. Vectors: Cosine Distance
$$s(\vec{x}, x') = \frac{x^t x'}{\|x\| \|x'\|}$$
2. Sets: Jaccard Distance
$$J(A, B) = \frac{|A \cap B|}{|A \cup B|} = \frac{|A \cap B|}{|A| + |B| - |A \cap B|}$$
 If sets A and B are both empty, then we define $J(A, B) = 1$ otherwise $0 \leq J(A, B) \leq 1$
3. Points: Euclidean Distance
$$d(x, x') = \left(\sum_{k=1}^p |x_k - x'_k|^q \right)^{1/q}$$

2.4 Clustering Algorithms

Clustering algorithms can be categorized into four types as listed below:

1. Exclusive Clustering
2. Overlapping Clustering
3. Hierarchical Clustering
4. Probabilistic Clustering

In exclusive clustering, data is clustered exclusively such that a point belonging to one cluster can not be included in another cluster.

Overlapping clustering uses fuzzy sets to cluster data such that a point can belong to more than one cluster but with varying levels of membership. We therefore group data according to an appropriate membership value.

In hierarchical clustering, every data point is treated as a cluster and at each iteration we find the union between the two nearest clusters until we reach the number of clusters we wanted.

Finally, a probabilistic approach clusters each point based on their probability of belonging to a particular cluster.

Now let's look at some of the most commonly used clustering algorithms:

- K-means
- Fuzzy K-means
- Hierarchical clustering
- Mixture of Gaussians

K-means is an example of an exclusive clustering algorithm. Fuzzy K-means is an example of an overlapping clustering algorithm. Hierarchical clustering is just that, an example of a Hierarchical clustering algorithm. A mixture of Gaussians is an example of a probabilistic clustering algorithm.

Let's get a better understanding of each clustering method above.

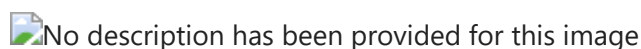
3. K-Means Clustering

K-means clustering works by classifying a dataset through a defined number of clusters that is fixed a priori.

Here is a walkthrough of the k-means algorithm:

1. We start by selecting k centroids where k is the number of clusters.
2. We then place the k centroids in our training data at different random places.
3. We then calculate the distance from each point in the training data to each centroid.
4. The points are then grouped based on the nearest centroid.
5. The grouped points are isolated together with their respective centroid. Find the mean of each group, then move the centroid to the location of the mean we have calculated.
6. We repeat the steps until we reach an optimal number of k groups.

Fig 2: A K-means clustering procedure flow chart



Source: Bustamam, A. et al. "Application of K-means Clustering Algorithm in Grouping the DNA Sequences of Hepatitis B Virus (HBV)." *AIP Conference Proceedings*, vol. 1862. no. 1. AIP Publishing LLC, 2017.

Mathematically, the objective of the K-means algorithm is to minimize an objective function which in our case is a squared error function given by $J = \sum_{j=1}^k \sum_{i=1}^n ||x_i - C_j||^2$ where

$\|x_i - C_j\|^2$ is the Euclidean distance between data points x_i and the centroid C_j .

Below is the mathematical walkthrough of the algorithm:

1. Let $X = \{x_1, x_2, \dots, x_n\}$ be our data points and $V = \{v_1, v_2, \dots, v_c\}$ be the centroids.
2. Select centroids randomly.
3. Then, calculate the distance between the centroids and each data point.
4. We then assign the data points to the centroids closest to them.
5. We recalculate the new centroids by computing the mean of the grouping, $v_i = \left(\frac{1}{c_i}\right) \sum_{j=1}^{c_i} x_j$ where c_i is the number of data points in the i th cluster.
6. Find the distance between the new centroid and each data point.
7. If there is no change in the composition of the clusters then we stop, or else we repeat the procedure from step 3.

The issue with the K-means algorithm is that the number of clusters need to be selected before we begin modeling. We can guess the number of clusters using either the Silhouette score or the elbow method.

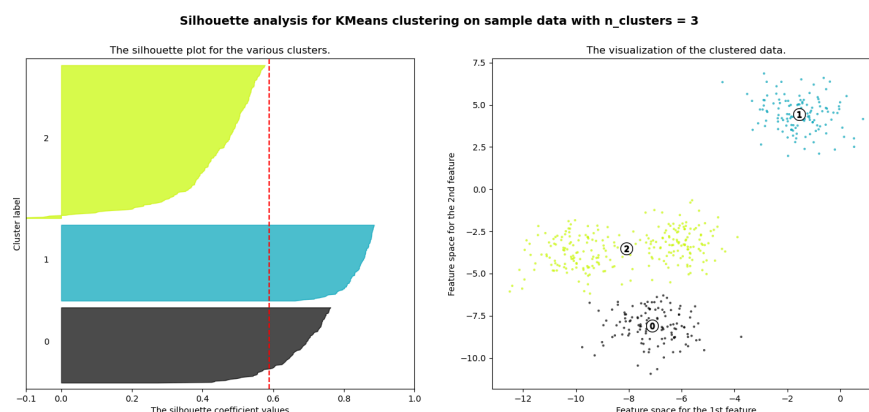
3.1 Silhouette Analysis

Silhouette analysis is applied in clustering to investigate the separation distance between our clusters. We use the Silhouette plot to visualize the distance of the points in one cluster to points in the neighboring clusters. The measure has a range of $[-1, 1]$.

Silhouette coefficients nearer to $+1$ show that the data points are far away from the adjacent clusters. A coefficient of 0 shows that the data points are very close to the neighboring clusters. Negative coefficients shows that the data points are probably assigned to the wrong cluster.

The Silhouette coefficient is given by $S = \frac{b-a}{\max(a,b)}$. An example of a Silhouette diagram is shown below.

Fig 3: Silhouette Analysis on Dataset with 3 Clusters



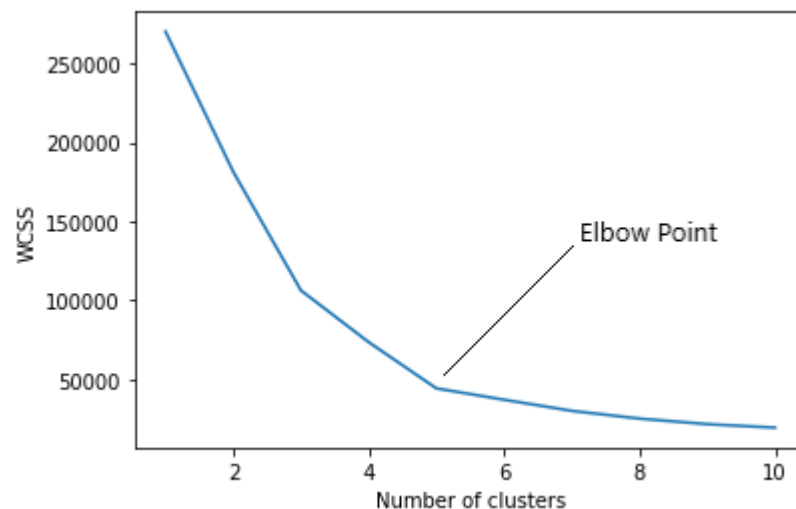
Source: [scikit-learn, Selecting the number of clusters with silhouette analysis on KMeans clustering](#)

3.2 Elbow Method

We can construct an elbow plot to find the appropriate value of k to use in our clustering.

We find the optimal number of clusters (groups) by drawing a line plot of Within the Sum of Squares (WSS) versus the number of clusters. We define WSS as the sum of square difference between each point in a cluster and its cluster center. An elbow plot is shown in the figure below:

Fig 4: An Elbow Plot Showing the Number of Optimal Clusters



Source: [Tripathi, Shreya et al. "Approaches to Clustering in Customer Segmentation." International Journal of Engineering & Technology, 2012, vol. 7, no. 3.12, 2018, pp. 802-807.](#)

Note that in the K-means algorithm, we do not have a general method for getting a unique optimal number of clusters.

We now put K-means into action with a simple example.

```
In [ ]: # Import packages

import matplotlib.pyplot as plt
import numpy as np
from sklearn.cluster import KMeans
from sklearn.datasets import make_blobs

np.random.seed(0)

%matplotlib inline
```

We generate a dataset with 4 clusters from the scikit-learn library 'make_blobs' and plot it for visualization.

```
In [ ]: X, y = make_blobs(centers=4, n_samples=2500)
```

```

fig = plt.figure(figsize=(8, 6))
plt.scatter(X[:, 0], X[:, 1], c=y)
plt.title("Dataset with 4 clusters", fontsize=20)
plt.xlabel("First feature")
plt.ylabel("Second feature")
plt.suptitle(
    "Fig. 5: Dataset Visualization", fontweight="bold", horizontalalignment="right"
)
plt.show()

```

In the plot above, we see that some clusters intersect with each other and this poses a beautiful problem for us to investigate and see if our algorithm will be able to distinguish them.

let's plot the elbow curve to establish if we will get the expected 4 clusters that we have generated.

```

In [ ]: wcss = []
        for i in range(1, 11):
            kmeans = KMeans(
                n_clusters=i, init="k-means++", max_iter=300, n_init=10, random_state=42
            )
            kmeans.fit(X)
            wcss.append(kmeans.inertia_)

plt.rcParams["figure.figsize"] = (7, 5)
plt.plot(range(1, 11), wcss)
plt.title("K-Means Clustering(The Elbow Method)", fontsize=20)
plt.xlabel("K")
plt.ylabel("WCSS")
plt.suptitle("Fig. 6: The Elbow Method", fontweight="bold", horizontalalignment="right")
plt.show()

```

Note that after the value of $k = 4$, the WCSS decreases very slowly. Therefore, our value of k is equal to 4.

```

In [ ]: kmeans = KMeans(n_clusters=4, init="k-means++", max_iter=300, n_init=10, random_state=42)
        ymeans = kmeans.fit_predict(X)

plt.rcParams["figure.figsize"] = (10, 10)
plt.title("Cluster of Dataset", fontsize=30)

plt.scatter(X[ymean == 0, 0], X[ymean == 0, 1], s=100, c="pink", label="Cluster 0")
plt.scatter(X[ymean == 1, 0], X[ymean == 1, 1], s=100, c="orange", label="Cluster 1")
plt.scatter(
    X[ymean == 2, 0], X[ymean == 2, 1], s=100, c="lightgreen", label="Cluster 2"
)
plt.scatter(X[ymean == 3, 0], X[ymean == 3, 1], s=100, c="red", label="Cluster 3")
plt.scatter(
    kmeans.cluster_centers[:, 0], kmeans.cluster_centers[:, 1], s=50, c="black"
)

plt.xlabel("First Feature")
plt.ylabel("Second Feature")
plt.legend()
# plt.suptitle('Fig 10: Clustering Results', fontsize=20, y = 1, ha = 'right')
plt.suptitle(

```

```
    "Fig. 7: Clustering Results", fontweight="bold", horizontalalignment="right"
)
plt.show()
```

We have trained the K-means algorithm in our dataset and we can see that the output shows clear distinction between different clusters.

In the next subsection, we will discuss the expectation-maximization approach of finding the composition of the clusters.

3.3 k-Means Algorithm: Expectation–Maximization

The Expectation-Maximization (E-M) procedure is as follows.

1. Randomly select the cluster centers.
2. Repeat the following steps until convergence is reached:
 - E-Step: Points are assigned to the nearest cluster center.
 - M-Step: Find the mean of each cluster and replace the cluster center with the mean.

We can visualize the E-M algorithm in action below.

```
In [ ]: from sklearn.datasets import make_blobs
        from sklearn.metrics import pairwise_distances_argmin

X, y_true = make_blobs(n_samples=300, centers=4, cluster_std=0.60, random_state=
rng = np.random.RandomState(42)
centers = [0, 4] + rng.randn(4, 2)

def draw_points(ax, c, factor=1):
    ax.scatter(X[:, 0], X[:, 1], c=c, cmap="viridis", s=50 * factor, alpha=0.3)

def draw_centers(ax, centers, factor=1, alpha=1.0):
    ax.scatter(
        centers[:, 0],
        centers[:, 1],
        c=np.arange(4),
        cmap="viridis",
        s=200 * factor,
        alpha=alpha,
    )
    ax.scatter(centers[:, 0], centers[:, 1], c="black", s=50 * factor, alpha=alp

def make_ax(fig, gs):
    ax = fig.add_subplot(gs)
    ax.xaxis.set_major_formatter(plt.NullFormatter())
    ax.yaxis.set_major_formatter(plt.NullFormatter())
    return ax

fig = plt.figure(figsize=(15, 4))
gs = plt.GridSpec(
```

```

    4, 15, left=0.02, right=0.98, bottom=0.05, top=0.95, wspace=0.2, hspace=0.2
)
ax0 = make_ax(fig, gs[:4, :4])
ax0.text(
    0.98,
    0.98,
    "Random Initialization",
    transform=ax0.transAxes,
    ha="right",
    va="top",
    size=16,
)
draw_points(ax0, "gray", factor=2)
draw_centers(ax0, centers, factor=2)

for i in range(3):
    ax1 = make_ax(fig, gs[:2, 4 + 2 * i : 6 + 2 * i])
    ax2 = make_ax(fig, gs[2:, 5 + 2 * i : 7 + 2 * i])

    # E-step
    y_pred = pairwise_distances_argmin(X, centers)
    draw_points(ax1, y_pred)
    draw_centers(ax1, centers)

    # M-step
    new_centers = np.array([X[y_pred == i].mean(0) for i in range(4)])
    draw_points(ax2, y_pred)
    draw_centers(ax2, centers, alpha=0.3)
    draw_centers(ax2, new_centers)
    for i in range(4):
        ax2.annotate(
            "",
            new_centers[i],
            centers[i],
            arrowprops=dict(arrowstyle="->", linewidth=1),
        )

    # Finish iteration
    centers = new_centers
    ax1.text(
        0.95, 0.95, "E-Step", transform=ax1.transAxes, ha="right", va="top", siz
    )
    ax2.text(
        0.95, 0.95, "M-Step", transform=ax2.transAxes, ha="right", va="top", siz
    )

# Final E-step
y_pred = pairwise_distances_argmin(X, centers)
axf = make_ax(fig, gs[:4, -4:])
draw_points(axf, y_pred, factor=2)
draw_centers(axf, centers, factor=2)
axf.text(
    0.98,
    0.98,
    "Final Clustering",
    transform=axf.transAxes,
    ha="right",
    va="top",
    size=16,

```

```
)
plt.suptitle(
    "Fig. 8: Expectation-Maximization", fontweight="bold", horizontalalignment="
)
plt.show()
```

The limitations of the E-M algorithm are:

- We may not achieve a globally optimal solution.
- We must choose the number of clusters before modeling.
- K-means will be slow for big data.
- K-means only works with linear cluster boundaries.

4. Fuzzy K-Means Clustering

Fuzzy clustering is the same as the K-means algorithm with the difference being that each point has some measure (probability) of belonging to each cluster in our dataset. In this case, a point might belong to 2 or more clusters if the point is at the middle of the clusters.

The other processes in K-means such as random initialization, iteration, and algorithm termination are the same in fuzzy K-means. As such, the clusters are treated as probabilistic distributions. By converting the probability function to a binary distribution giving 1 for data points close to a centroid and 0 otherwise, the fuzzy K-means becomes the usual K-means.

Below is the procedure for the fuzzy K-means algorithm;

- First, fix the number of clusters.
- Initialize the centroids randomly and compute the probability of the data points belonging to the different clusters.
- Knowing the probability of each data point belonging to a cluster, we recalculate new centroids by computing the mean of the clusters.
$$v_k(n+1) = \frac{\sum_{x_i \in k} x_i P(v_k|x_i)^b}{\sum_{x_i \in k} P(v_k|x_i)^b}$$
- The algorithm is terminated when it converges or when we reach the number of specified iterations.

5. Hierarchical Clustering

In hierarchical clustering we define a distance matrix using linkage with the following parameters

- Method: To calculate the distance between clusters.
- Metric: The distance metric to be used.
- Optimal ordering: To order data points.

The different types of methods we use in hierarchical clustering are:

1. Single: Finds the distance between the closest points in different clusters.

2. Complete: Finds the distance between the farthest points in different clusters.
3. Average: Finds the distance between the arithmetic mean of our clusters.
4. Centroids: Finds the distance between the geometric mean of our clusters.
5. Median: Finds the distance between the median of our clusters.
6. Ward: Finds distance based on our sum of squares.

Below are Python scripts on how to implement the different methods.

5.1 Ward Method

```
In [ ]: import pandas as pd
        from sklearn.datasets import make_blobs

        X1, y1 = make_blobs(
            n_samples=50, centers=[[4, 4], [-2, -1], [1, 1], [10, 4]], cluster_std=0.9
        )
        df = pd.DataFrame(X1, columns=["X", "y"])
        df.head()
```

```
In [ ]: import seaborn as sns
        from scipy.cluster.hierarchy import fcluster, linkage

        # Use the Linkage()
        distance_matrix = linkage(df[["X", "y"]], method="ward", metric="euclidean")

        # Assign cluster labels
        df["cluster_labels"] = fcluster(distance_matrix, 2, criterion="maxclust")

        # Plot clusters
        sns.scatterplot(x="X", y="y", hue="cluster_labels", data=df)
        plt.suptitle("Fig. 9: Ward Method", fontweight="bold", horizontalalignment="right")
        plt.show();
```

5.2 Single Method

```
In [ ]: # Use the Linkage()
        distance_matrix = linkage(df[["X", "y"]], method="single", metric="euclidean")

        # Assign cluster labels
        df["cluster_labels_2"] = fcluster(distance_matrix, 2, criterion="maxclust")

        # Plot clusters
        sns.scatterplot(x="X", y="y", hue="cluster_labels_2", data=df)
        plt.suptitle("Fig. 10: Single Method", fontweight="bold", horizontalalignment="right")
        plt.show();
```

5.3 Complete Method

```
In [ ]: # Use the Linkage()
        distance_matrix = linkage(df[["X", "y"]], method="complete", metric="euclidean")

        # Assign cluster labels
        df["cluster_labels_3"] = fcluster(distance_matrix, 2, criterion="maxclust")
```

```
# Plot clusters
sns.scatterplot(x="X", y="y", hue="cluster_labels", data=df)
plt.suptitle("Fig. 11: Complete Method", fontweight="bold", horizontalalignment="right")
plt.show();
```

5.4 Dendrogram

Dendrograms help us visualize how clusters are formed, and from them, we can choose the number of clusters in our dataset. Below is simple code that gives us a dendrogram diagram.

```
In [ ]: from scipy.cluster.hierarchy import dendrogram

# Create a dendrogram
dn = dendrogram(distance_matrix)
plt.suptitle("Fig. 11: Dendrogram", fontweight="bold", horizontalalignment="right")
plt.show()
```

In summary, the following are steps taken by hierarchical clustering:

1. We treat each point in our dataset as its own cluster.
2. We then calculate the Euclidean distance between the centroids of all clusters in our dataset.
3. We join the closest clusters together.
4. We repeat steps 2 and 3 until we get only one cluster that contains all the data points.
5. A dendrogram that shows the evolution of the hierarchical structure is plotted showing how clusters are arranged from top to bottom.
6. It is now up to us to decide at which level we want to create our clusters.

6. Gaussian Mixture Models (GMM)

This is a model-based approach that uses several models to generate clusters and attempts to optimize the model fit on the data.

The advantages of this type of clustering includes that the possibility of one point belonging to several clusters, a density approximation for each cluster, the availability of robust statistical inference methods to do our clustering, and the freedom to choose the component distribution of the model.

The K component distributions that make up a mixture model are collectively combined to give $f(x) = \sum_{k=1}^K a_k f_k(x)$ where a_k is the k -th component contribution in building $f(x)$.

Like the K-means algorithm above, GMM is also used to find the number of clusters in a dataset. Consider the example below.

```
In [ ]: %matplotlib inline
import matplotlib.pyplot as plt
import numpy as np
```

```
import seaborn as sns

# Generate some data
from sklearn.datasets import make_blobs

sns.set()

X, y_true = make_blobs(n_samples=400, centers=4, cluster_std=0.60, random_state=
X = X[:, ::-1] # flip axes for better plotting
```

To visualize the number of clusters, we fit our Gaussian mixture model with a pre-specified number of clusters.

```
In [ ]: from sklearn import mixture

gmm = mixture.GaussianMixture(n_components=4).fit(X)
labels = gmm.predict(X)
plt.scatter(X[:, 0], X[:, 1], c=labels, s=40, cmap="viridis")
plt.suptitle(
    "Fig. 12: Clusters from GMM", fontweight="bold", horizontalalignment="right"
)
plt.show();
```

The Gaussian mixture model contains a probabilistic model that gives the probability of a point belonging to a cluster.

```
In [ ]: probs = gmm.predict_proba(X)
print(probs[:5].round(3))
```

The Gaussian mixture model like the k-means algorithm uses the expectation-maximization approach below.

1. We choose a random starting point.
2. Repeat the following until convergence.
 - E-Step: Find the weight of each point belonging to a given cluster.
 - M-Step: Update the location and shape of each cluster based on the data point weights.

GMM will at times miss the globally optimal solution, and it is therefore recommended to use different random initialization.

6.1 Number of Clusters

GMM being a generative model will provide us with a natural technique of finding the optimal number of clusters. Since a generative model provides probability distribution for a given dataset, we can use analytic criterion techniques like *Akaike Information Criterion (AIC)* or *Bayesian Information Criterion (BIC)* to reduce the chances of our model overfitting. We can also use cross-validation to achieve the same.

In the example below, we use AIC and BIC.

```
In [ ]: from sklearn.datasets import make_moons

Xmoon, ymoon = make_moons(200, noise=0.05, random_state=0)
plt.scatter(Xmoon[:, 0], Xmoon[:, 1])

n_components = np.arange(1, 21)
models = [
    mixture.GaussianMixture(n, covariance_type="full", random_state=0).fit(Xmoon)
    for n in n_components
]

plt.plot(n_components, [m.bic(Xmoon) for m in models], label="BIC")
plt.plot(n_components, [m.aic(Xmoon) for m in models], label="AIC")
plt.legend(loc="best")
plt.xlabel("n_components")
plt.suptitle(
    "Fig. 13: Optimal Clusters", fontweight="bold", horizontalalignment="right"
);
```

7. DBSCAN

DBSCAN is short for Density-Based Spatial Clustering of Applications with Noise. It works by identifying regions that are densely populated from sparsely populated regions. The logic follows that points belonging to a cluster should be close to the points in the same cluster.

It is defined by two parameters:

1. **Epsilon** that finds the radius, which has enough points within.
2. **Minimum Samples** that finds the minimum number of points we want to be in a cluster.

Below is an implementation of the DBSCAN algorithm.

```
In [ ]: import matplotlib.pyplot as plt
import numpy as np
from sklearn.cluster import DBSCAN
from sklearn.datasets import make_blobs
from sklearn.preprocessing import StandardScaler

%matplotlib inline
```

```
In [ ]: def createDataPoints(centroidLocation, numSamples, clusterDeviation):
    # Create random data and store in feature matrix X and response vector y.
    X, y = make_blobs(
        n_samples=numSamples, centers=centroidLocation, cluster_std=clusterDeviation
    )

    # Standardize features by removing the mean and scaling to unit variance
    X = StandardScaler().fit_transform(X)
    return X, y
```

```
In [ ]: X, y = createDataPoints([[4, 3], [2, -1], [-1, 4]], 1500, 0.5)
```

```

In [ ]: epsilon = 0.3
        minimumSamples = 7
        db = DBSCAN(eps=epsilon, min_samples=minimumSamples).fit(X)
        labels = db.labels_
        labels

In [ ]: # First, create an array of booleans using the labels from db.
        core_samples_mask = np.zeros_like(db.labels_, dtype=bool)
        core_samples_mask[db.core_sample_indices_] = True
        core_samples_mask

In [ ]: # Number of clusters in labels, ignoring noise if present.
        n_clusters_ = len(set(labels)) - (1 if -1 in labels else 0)
        n_clusters_

In [ ]: # Remove repetition in labels by turning it into a set.
        unique_labels = set(labels)
        unique_labels

In [ ]: # Create colors for the clusters.
        colors = plt.cm.Spectral(np.linspace(0, 1, len(unique_labels)))
        colors

In [ ]: # Plot the points with colors
        for k, col in zip(unique_labels, colors):
            if k == -1:
                # Black used for noise.
                col = "k"

            class_member_mask = labels == k

            # Plot the datapoints that are clustered
            xy = X[class_member_mask & core_samples_mask]
            plt.scatter(xy[:, 0], xy[:, 1], s=50, c=col, marker="o", alpha=0.5)

            # Plot the outliers
            xy = X[class_member_mask & ~core_samples_mask]
            plt.scatter(xy[:, 0], xy[:, 1], s=50, c=col, marker="o", alpha=0.5)
        plt.show()

```

8. Conclusion

We have introduced the concept of clustering and have given an overview of the clustering techniques applied to a dataset. We also illustrated a simple application of K-means clustering to datasets. In the next lesson, we are going to apply hierarchical clustering in a real world dataset.

References

1. Chang, Chih-Tang, Jim ZC Lai, and Mu-Der Jeng. "A fuzzy k-means clustering algorithm using cluster center displacement." *J. Inf. Sci. Eng.* 27.3 (2011): 995-1009.
2. Cohn, Ryan, and Elizabeth Holm. "Unsupervised Machine Learning via Transfer Learning and k-means Clustering to Classify Materials Image Data." *Integrating*

- Materials and Manufacturing Innovation*, vol. 10, no. 2, 2021, pp. 231-244.
3. Davis, Damek, Mateo Diaz, and Kaizheng Wang. "Clustering a mixture of gaussians with unknown covariance." arXiv preprint arXiv:2110.01602 (2021).
 4. Ghahramani, Zoubin. "Unsupervised Learning." *Advanced Lectures on Machine Learning*, edited by Oliver Bousquet, Ulrike von Luxburg, and Gunnar Rätsch, Springer, 2003, pp. 72-112.
 5. Goldberger, Jacob, and Sam Roweis. "Hierarchical Clustering of a Mixture Model." *Advances in Neural Information Processing Systems*, vol. 17, 2004.
 6. Murtagh, Fionn, and Pedro Contreras. "Algorithms for Hierarchical Clustering: An Overview." *Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery*, vol. 2, no. 1, 2012, pp. 86-97.
 7. Nielsen, Frank. *Introduction to HPC with MPI for Data Science*. Springer, 2016.
-

MACHINE LEARNING IN FINANCE

MODULE 2 | LESSON 2

HIERARCHICAL CLUSTERING

Reading Time	70 minutes
Prior Knowledge	Clustering, unsupervised
Keywords	dendrogram, hierarchical

In the previous lesson, we explored unsupervised machine learning and saw how it differs from supervised machine learning. We also studied different clustering techniques like the k-means algorithm to find patterns in a dataset. In this lesson, we will go further using hierarchical clustering and implement it in clustering foreign exchange rates.

1. Introduction

Hierarchical clustering is an unsupervised learning model that clusters data points into hierarchies. In the last lesson, we saw how the k-means algorithm operates, and one of its limitations was that we need to know the number of clusters beforehand. In hierarchical clustering, it is not necessary to know the number of clusters before the modeling process can begin.

Hierarchical clustering can be divided into two types:

1. Agglomerative hierarchical clustering
2. Divisive hierarchical clustering

2. Agglomerative Hierarchical Clustering

Agglomerative hierarchical clustering is the most widely used (in the hierarchical family) to cluster data points based on their similarity. It uses a bottom-up approach where each data point starts as an individual cluster, and at each iteration, similar clusters are merged as we move up the hierarchy until we obtain one cluster or we identify K clusters.

2.1 Algorithm

1. Assume we have the dataset scattered in a two dimensional plane.
2. Each data point forms its own cluster at the beginning.

3. Similar clusters are merged.

4. Repeat step 2 above until a single cluster or K clusters remain(s).

Two clusters are considered similar if the distance between them is small.

2.2 Measures of Distance (Similarity)

Euclidean distance is defined as the distance between two points in a straight line. For example, if (x_1, y_1) and (x_2, y_2) are the points in a 2-dimensional space, then the Euclidean distance would be $\text{Euclidean Distance} = (x_2 - x_1)^2 + (y_2 - y_1)^2$. Euclidean Distance is applied when the variables are continuous.

Other methods of measuring distance between two points include;

- Hamming distance

We use hamming distance for text and non-numerical data, and it works by counting the number of times the coordinates in the two sets differ.

- Manhattan distance

Manhattan distance is suitable when finding the distance between different locations like finding the distance between schools. It is given by $|a-b|_1 = \sum_i |a_i - b_i|$

- Minkowski distance

Minkowski distance is given by $\left(\sum_{i=1}^n |X_i - Y_i|^p\right)^{1/p}$. When $p=1$ the Minkowski distance is equivalent to Manhattan distance, for $p=2$ the Minkowski distance is equivalent to Euclidean distance.

- Maximum distance

Maximum distance is the maximum distance we can get between two sets, that is, $|a-b|_{\infty} = \max_i |a_i - b_i|$

- Canberra distance

Canberra distance is expressed as $\sum_i \frac{|x_i - y_i|}{|x_i + y_i|}$ and terms where the denominator is zero are omitted. Canberra distance is used to compare ranked lists.

The choice of distance metric to use depends on the problem we are trying to solve.

2.3 Linkage Criterion

Once we have settled on a distance metric to apply, we need to decide where the distance will be computed. This is referred to as **linkage**.

The commonly used linkage methods are:

1. Single linkage / nearest linkage:

This is the minimum distance between a pair of data points in two different clusters given by $d(u,v) = \min(\text{dist}(u[i], v[j]))$ for data points i in cluster u and data points j in cluster v . Consider the example below

```
In [ ]: from matplotlib import pyplot as plt
        from scipy.cluster.hierarchy import dendrogram, linkage

        X = [[i] for i in [2, 8, 0, 4, 1, 9, 9, 0]]
```

From the given dataset, we obtain a single linkage using the code below.

```
In [ ]: Z = linkage(X, "single")
        fig = plt.figure(figsize=(25, 5))
        plt.suptitle(
            "Fig. 1: Dendrogram with Single Linkage",
            fontweight="bold",
            horizontalalignment="right",
        )
        dn = dendrogram(Z)
```

2. Complete Linkage:

This is the maximum distance we can get between data points in two different clusters. The mathematical representation for this is $d(u,v) = \max(\text{dist}(u[i], v[j]))$ for data points i in cluster u and data points j in cluster v . We define the complete linkage in a code as shown below:

```
In [ ]: Z = linkage(X, "complete")
        fig = plt.figure(figsize=(25, 5))
        plt.suptitle(
            "Fig. 2: Dendrogram with Complete Linkage",
            fontweight="bold",
            horizontalalignment="right",
        )
        dn = dendrogram(Z)
```

3. Average Linkage:

This is the average of the distance between each data point in one cluster to the data point in the next cluster. Its mathematical representation is as shown below $d(u,v) = \sum_{i,j} \frac{\text{dist}(u[i], v[j])}{(|u| \times |v|)}$

and just like the two linkages above we specify linkage as average in python code.

```
In [ ]: Z = linkage(X, "average")
        fig = plt.figure(figsize=(25, 5))
        plt.suptitle(
            "Fig. 3: Dendrogram with Average Linkage",
            fontweight="bold",
            horizontalalignment="right",
        )
        dn = dendrogram(Z)
```

4. Centroid Linkage:

This is the distance between the cluster centers and is given by $d(u,v) = \|c_u - c_v\|_2$ with the linkage specified as in the preceding charts.

```
In [ ]: Z = linkage(X, "centroid")
fig = plt.figure(figsize=(25, 5))
plt.suptitle(
    "Fig. 4: Dendrogram with Centroid Linkage",
    fontweight="bold",
    horizontalalignment="right",
)
dn = dendrogram(Z)
```

5. Ward's Linkage:

Ward's linkage minimizes the increase in the sum of square error at each iteration.

```
In [ ]: Z = linkage(X, "ward")
fig = plt.figure(figsize=(25, 5))
plt.suptitle(
    "Fig. 5: Dendrogram with Ward Linkage",
    fontweight="bold",
    horizontalalignment="right",
)
dn = dendrogram(Z)
```

The diagrams we have seen above are called Dendrograms and we chose the number of clusters in hierarchical clustering with the help of a dendrogram.

2.4 Dendrogram

A dendrogram is a diagram that shows the hierarchical relationship between clusters. It shows how clusters are formed at each step.

Once we have drawn the dendrogram, we slice it horizontally at the desired level then the branches that appear below the cut form individual clusters and its associated membership. Below is an example of a dendrogram with horizontal slices at different levels.

```
In [ ]: Z = linkage(X)
fig = plt.figure(figsize=(25, 5))
plt.axhline(1.2, color="red", linestyle="--")
plt.axhline(2.2, color="red", linestyle="--")
plt.axhline(0.8, color="red", linestyle="--")
plt.suptitle(
    "Fig. 6: Dendrogram with Different Horizontal Lines",
    fontweight="bold",
    horizontalalignment="right",
)
dn = dendrogram(Z)
```

3. Divisive Hierarchical Clustering

Divisive hierarchical clustering works in the opposite of agglomerative hierarchical clustering; that is, we start with a single cluster containing all the data points. Then, at each iteration, we split the data into smaller clusters until we have each data point in its own cluster.

4. Implementation of Hierarchical Clustering

In this demonstration, we use a matrix of daily returns of foreign exchange rates and perform hierarchical clustering to determine if the currencies form clusters that are related to their geographical categories.

4.1 Data Scraping

We start by installing the Yahoo finance API that will be used to download the forex data. We also import libraries for data manipulation and modeling.

```
In [ ]: # For data manipulation
# For visualization
import matplotlib.pyplot as plt
import pandas as pd
import seaborn as sns

# To fetch financial data
import yfinance as yf

plt.style.use("seaborn-darkgrid")
%matplotlib inline

from scipy.cluster.hierarchy import cophenet, dendrogram, linkage

# to perform hierarchical clustering, compute cophenetic correlation, and create
from sklearn.cluster import AgglomerativeClustering

sns.set_theme()

# Removes the limit for the number of displayed columns
pd.set_option("display.max_columns", None)

# Sets the limit for the number of displayed rows
pd.set_option("display.max_rows", 200)

# to compute distances
from scipy.spatial.distance import pdist

# to scale the data using z-score
from sklearn.preprocessing import StandardScaler
```

We now download the foreign exchange rates rendered to one U.S. dollar.

```
In [ ]: # Set the ticker as 'EURUSD=X'
forex_data = yf.download("USDEUR=X", start="2019-01-02", end="2022-06-30")
forex_data = forex_data.reset_index()
euro_df = forex_data[["Date", "Adj Close"]]
euro_df.rename(columns={"Adj Close": "euro"}, inplace=True)
```

```
# Set the index to a datetime object
```

```
forex_data1 = yf.download("USDRUB=X", start="2019-01-02", end="2022-06-30")
forex_data1 = forex_data1.reset_index()
rub_df = forex_data1[["Date", "Adj Close"]]
rub_df.rename(columns={"Adj Close": "rub"}, inplace=True)
```

```
forex_data2 = yf.download("USDGBP=X", start="2019-01-02", end="2022-06-30")
forex_data2 = forex_data2.reset_index()
gbp_df = forex_data2[["Date", "Adj Close"]]
gbp_df.rename(columns={"Adj Close": "gbp"}, inplace=True)
```

```
forex_data3 = yf.download("USDJPY=X", start="2019-01-02", end="2022-06-30")
forex_data3 = forex_data3.reset_index()
jpy_df = forex_data3[["Date", "Adj Close"]]
jpy_df.rename(columns={"Adj Close": "jpy"}, inplace=True)
```

```
forex_data4 = yf.download("USDKES=X", start="2019-01-02", end="2022-06-30")
forex_data4 = forex_data4.reset_index()
kes_df = forex_data4[["Date", "Adj Close"]]
kes_df.rename(columns={"Adj Close": "kes"}, inplace=True)
```

```
forex_data5 = yf.download("USDCNY=X", start="2019-01-02", end="2022-06-30")
forex_data5 = forex_data5.reset_index()
cny_df = forex_data5[["Date", "Adj Close"]]
cny_df.rename(columns={"Adj Close": "cny"}, inplace=True)
```

```
forex_data6 = yf.download("USDKRW=X", start="2019-01-02", end="2022-06-30")
forex_data6 = forex_data6.reset_index()
krw_df = forex_data6[["Date", "Adj Close"]]
krw_df.rename(columns={"Adj Close": "krw"}, inplace=True)
```

```
forex_data7 = yf.download("USDSGD=X", start="2019-01-02", end="2022-06-30")
forex_data7 = forex_data7.reset_index()
sgd_df = forex_data7[["Date", "Adj Close"]]
sgd_df.rename(columns={"Adj Close": "sgd"}, inplace=True)
```

```
forex_data8 = yf.download("USDTHW=X", start="2019-01-02", end="2022-06-30")
forex_data8 = forex_data8.reset_index()
thw_df = forex_data8[["Date", "Adj Close"]]
thw_df.rename(columns={"Adj Close": "thw"}, inplace=True)
```

```
forex_data9 = yf.download("USDNGN=X", start="2019-01-02", end="2022-06-30")
forex_data9 = forex_data9.reset_index()
ngn_df = forex_data9[["Date", "Adj Close"]]
ngn_df.rename(columns={"Adj Close": "ngn"}, inplace=True)
```

```
forex_data10 = yf.download("USDZAR=X", start="2019-01-02", end="2022-06-30")
forex_data10 = forex_data10.reset_index()
zar_df = forex_data10[["Date", "Adj Close"]]
zar_df.rename(columns={"Adj Close": "zar"}, inplace=True)
```

```
forex_data11 = yf.download("USDMYR=X", start="2019-01-02", end="2022-06-30")
forex_data11 = forex_data11.reset_index()
myr_df = forex_data11[["Date", "Adj Close"]]
myr_df.rename(columns={"Adj Close": "myr"}, inplace=True)
```

```
forex_data12 = yf.download("USDIDR=X", start="2019-01-02", end="2022-06-30")
forex_data12 = forex_data12.reset_index()
idr_df = forex_data12[["Date", "Adj Close"]]
```

```
idr_df.rename(columns={"Adj Close": "idr"}, inplace=True)

forex_data13 = yf.download("USDTHB=X", start="2019-01-02", end="2022-06-30")
forex_data13 = forex_data13.reset_index()
thb_df = forex_data13[["Date", "Adj Close"]]
thb_df.rename(columns={"Adj Close": "thb"}, inplace=True)

forex_data14 = yf.download("USDAUD=X", start="2019-01-02", end="2022-06-30")
forex_data14 = forex_data14.reset_index()
aud_df = forex_data14[["Date", "Adj Close"]]
aud_df.rename(columns={"Adj Close": "aud"}, inplace=True)

forex_data15 = yf.download("USDNZD=X", start="2019-01-02", end="2022-06-30")
forex_data15 = forex_data15.reset_index()
nzd_df = forex_data15[["Date", "Adj Close"]]
nzd_df.rename(columns={"Adj Close": "nzd"}, inplace=True)

forex_data16 = yf.download("USDCAD=X", start="2019-01-02", end="2022-06-30")
forex_data16 = forex_data16.reset_index()
cad_df = forex_data16[["Date", "Adj Close"]]
cad_df.rename(columns={"Adj Close": "cad"}, inplace=True)

forex_data17 = yf.download("USDCHF=X", start="2019-01-02", end="2022-06-30")
forex_data17 = forex_data17.reset_index()
chf_df = forex_data17[["Date", "Adj Close"]]
chf_df.rename(columns={"Adj Close": "chf"}, inplace=True)

forex_data18 = yf.download("USDNOK=X", start="2019-01-02", end="2022-06-30")
forex_data18 = forex_data18.reset_index()
nok_df = forex_data18[["Date", "Adj Close"]]
nok_df.rename(columns={"Adj Close": "nok"}, inplace=True)

forex_data19 = yf.download("USDAUD=X", start="2019-01-02", end="2022-06-30")
forex_data19 = forex_data19.reset_index()
sek_df = forex_data19[["Date", "Adj Close"]]
sek_df.rename(columns={"Adj Close": "sek"}, inplace=True)

forex_data20 = yf.download("USDARS=X", start="2019-01-02", end="2022-06-30")
forex_data20 = forex_data20.reset_index()
ars_df = forex_data20[["Date", "Adj Close"]]
ars_df.rename(columns={"Adj Close": "ars"}, inplace=True)

forex_data21 = yf.download("USDPLN=X", start="2019-01-02", end="2022-06-30")
forex_data21 = forex_data21.reset_index()
pln_df = forex_data21[["Date", "Adj Close"]]
pln_df.rename(columns={"Adj Close": "pln"}, inplace=True)

forex_data22 = yf.download("USDPHP=X", start="2019-01-02", end="2022-06-30")
forex_data22 = forex_data22.reset_index()
php_df = forex_data22[["Date", "Adj Close"]]
php_df.rename(columns={"Adj Close": "php"}, inplace=True)

forex_data23 = yf.download("USDRON=X", start="2019-01-02", end="2022-06-30")
forex_data23 = forex_data23.reset_index()
ron_df = forex_data23[["Date", "Adj Close"]]
ron_df.rename(columns={"Adj Close": "ron"}, inplace=True)

forex_data24 = yf.download("USDHUF=X", start="2019-01-02", end="2022-06-30")
forex_data24 = forex_data24.reset_index()
huf_df = forex_data24[["Date", "Adj Close"]]
```

```

huf_df.rename(columns={"Adj Close": "huf"}, inplace=True)

forex_data25 = yf.download("USDBRL=X", start="2019-01-02", end="2022-06-30")
forex_data25 = forex_data25.reset_index()
brl_df = forex_data25[["Date", "Adj Close"]]
brl_df.rename(columns={"Adj Close": "brl"}, inplace=True)

forex_data26 = yf.download("USDCLP=X", start="2019-01-02", end="2022-06-30")
forex_data26 = forex_data26.reset_index()
clp_df = forex_data26[["Date", "Adj Close"]]
clp_df.rename(columns={"Adj Close": "clp"}, inplace=True)

forex_data27 = yf.download("USDMXN=X", start="2019-01-02", end="2022-06-30")
forex_data27 = forex_data27.reset_index()
mxn_df = forex_data27[["Date", "Adj Close"]]
mxn_df.rename(columns={"Adj Close": "mxn"}, inplace=True)

forex_data28 = yf.download("USDCOP=X", start="2019-01-02", end="2022-06-30")
forex_data28 = forex_data28.reset_index()
cop_df = forex_data28[["Date", "Adj Close"]]
cop_df.rename(columns={"Adj Close": "cop"}, inplace=True)

forex_data29 = yf.download("USDILS=X", start="2019-01-02", end="2022-06-30")
forex_data29 = forex_data29.reset_index()
ils_df = forex_data29[["Date", "Adj Close"]]
ils_df.rename(columns={"Adj Close": "ils"}, inplace=True)

forex_data30 = yf.download("USDTRY=X", start="2019-01-02", end="2022-06-30")
forex_data30 = forex_data30.reset_index()
try_df = forex_data30[["Date", "Adj Close"]]
try_df.rename(columns={"Adj Close": "try"}, inplace=True)

forex_data31 = yf.download("USDINR=X", start="2019-01-02", end="2022-06-30")
forex_data31 = forex_data31.reset_index()
inr_df = forex_data31[["Date", "Adj Close"]]
inr_df.rename(columns={"Adj Close": "inr"}, inplace=True)

```

The currencies are combined into one dataframe that will form the basis of our modeling.

```

In [ ]: from functools import reduce

df_currencies = reduce(
    lambda x, y: pd.merge(x, y, on="Date", how="outer"),
    [
        kes_df,
        ars_df,
        php_df,
        myr_df,
        ils_df,
        cop_df,
        euro_df,
        ngn_df,
        huf_df,
        ron_df,
        cny_df,
        rub_df,
        clp_df,
        sgd_df,
        twd_df,

```

```

        krw_df,
        idr_df,
        thb_df,
        inr_df,
        pln_df,
        try_df,
        brl_df,
        mxn_df,
        zar_df,
        gbp_df,
        jpy_df,
        aud_df,
        nzd_df,
        cad_df,
        chf_df,
        nok_df,
        sek_df,
    ],
)
df_currencies.set_index("Date", inplace=True)

df_currencies.head(10)

```

We have 911 rows of daily rates and 32 currencies in our dataframe.

```
In [ ]: df_currencies.shape
```

```
In [ ]: df_currencies.info()
```

```
In [ ]: df_currencies.describe()
```

The next step is to find missing values in our dataset.

```
In [ ]: df_currencies.isna().sum()
```

We impute the missing value by propagating the observations from the last day to the missing day data.

```
In [ ]: df_currencies.fillna(method="ffill", inplace=True)
df_currencies.isna().sum()
```

Since the currencies differ a lot in value range, we scale them to ensure they have the same mean and unit variance.

```
In [ ]: # Scaling the data to bring it to the same scale
```

```

sc = StandardScaler()
subset_scaled_df = pd.DataFrame(
    sc.fit_transform(df_currencies),
    columns=df_currencies.columns,
)
subset_scaled_df.head()

```

```
In [ ]: # Generate simple percentage returns to calculate the respective betas
```

```
df_returns = (df_currencies / df_currencies.shift(-1)) - 1
df_returns.head()
```

4.2 Model Training

```
In [ ]: subset_scaled_df.drop(subset_scaled_df.tail(1).index, inplace=True)
```

```
In [ ]: subset_scaled_df.tail()
```

We now find the **cophenetic correlation** using a combination of distance metrics and linkage methods. A cophenetic correlation measures the dendrogram's ability to preserve the initial unmodeled data points. The higher the cophenetic correlation, the better the performance of a dendrogram in preserving the original distance.

The cophenetic correlation is defined as:
$$c = \frac{\sum_{i < j} (Y_{ij} - y)(Z_{ij} - z)}{\sum_{i < j} (Y_{ij} - y) \sum_{i < j} (Z_{ij} - z)}$$

where

- Y_{ij} is the distance between i and j in Y .
- Z_{ij} is the cophenetic distance between i and j .
- y and z are the mean of Y and Z respectively.

```
In [ ]: # list of distance metrics
distance_metrics = ["euclidean", "chebyshev", "cityblock"]

# list of linkage methods
linkage_methods = ["single", "complete", "average", "weighted"]

high_cophenet_corr = 0
high_dm_lm = [0, 0]

for dm in distance_metrics:
    for lm in linkage_methods:
        Z = linkage(subset_scaled_df.T, metric=dm, method=lm)
        c, coph_dists = cophenet(Z, pdist(subset_scaled_df.T))
        print(
            "Cophenetic correlation for {} distance and {} linkage is {}".format(
                dm.capitalize(), lm, c
            )
        )
        if high_cophenet_corr < c:
            high_cophenet_corr = c
            high_dm_lm[0] = dm
            high_dm_lm[1] = lm
```

We now identify the distance metric and linkage method that give the highest cophenetic correlation.

```
In [ ]: # printing the combination of distance metric and linkage method with the highest
print(
    "Highest cophenetic correlation is {}, which is obtained with {} distance an
    high_cophenet_corr, high_dm_lm[0].capitalize(), high_dm_lm[1]
```

```
)
)
```

Since the Euclidean distance gives us the best performance, we combine it with different linkage methods and evaluate which combination will improve our performance.

```
In [ ]: # List of linkage methods
linkage_methods = ["single", "complete", "average", "centroid", "ward", "weighted"]

high_cophenet_corr = 0
high_dm_lm = [0, 0]

for lm in linkage_methods:
    Z = linkage(subset_scaled_df.T, metric="euclidean", method=lm)
    c, coph_dists = cophenet(Z, pdist(subset_scaled_df.T))
    print("Cophenetic correlation for {} linkage is {}".format(lm, c))
    if high_cophenet_corr < c:
        high_cophenet_corr = c
        high_dm_lm[0] = "euclidean"
        high_dm_lm[1] = lm
```

Again, we print the linkage method that gives the highest cophenetic correlation.

```
In [ ]: # printing the combination of distance metric and linkage method with the highest
print(
    "Highest cophenetic correlation is {}, which is obtained with {} linkage".format(
        high_cophenet_corr, high_dm_lm[1]
    )
)
```

Let us plot the graphs of different dendrograms to visualize how clusters vary with different linkage methods.

```
In [ ]: # List of linkage methods
linkage_methods = ["single", "complete", "average", "centroid", "ward", "weighted"]

# Lists to save results of cophenetic correlation calculation
compare_cols = ["Linkage", "Cophenetic Coefficient"]

# to create a subplot image
fig, axs = plt.subplots(len(linkage_methods), 1, figsize=(15, 30))
# `plt.suptitle`()

# We will enumerate through the list of linkage methods above
# For each linkage method, we will plot the dendrogram and calculate the cophenetic correlation
for i, method in enumerate(linkage_methods):
    Z = linkage(subset_scaled_df.T, metric="euclidean", method=method)

    dendrogram(Z, ax=axs[i])
    axs[i].set_title(f"Dendrogram ({method.capitalize()} Linkage)")
    fig.suptitle(
        "Fig. 11: Dendrograms Plots", fontweight="bold", horizontalalignment="right"
    )

    coph_corr, coph_dist = cophenet(Z, pdist(subset_scaled_df.T))
    axs[i].annotate(
        f"Cophenetic\nCorrelation\n{coph_corr:0.2f}",

```

```
(0.80, 0.80),
xycoords="axes fraction",
)
```

To find the components of each cluster, we can train the data as shown below, then find the components as demonstrated in the steps below.

```
In [ ]: HCmodel = AgglomerativeClustering(n_clusters=3, affinity="euclidean", linkage="a
HCmodel.fit(subset_scaled_df.T)
```

```
In [ ]: df_currencies_t = df_currencies.T
subset_scaled_df_t = subset_scaled_df.T

subset_scaled_df_t["HC_Clusters"] = HCmodel.labels_
df_currencies_t["HC_Clusters"] = HCmodel.labels_
```

```
In [ ]: cluster_profile = df_currencies_t.groupby("HC_Clusters").mean()
```

```
In [ ]: cluster_profile.head()
```

```
In [ ]: cluster_profile
```

We finally see the composition of 3 different clusters and the outlier which will be the column 'HC Cluster' that was engineered above.

```
In [ ]: # Let's see the names of the securities in each cluster
for cl in df_currencies_t["HC_Clusters"].unique():
    print(
        "The",
        df_currencies_t[df_currencies_t["HC_Clusters"] == cl].iloc[:, 0].nunique
        "Securities in cluster",
        cl,
        "are:",
    )
    print(df_currencies_t[df_currencies_t["HC_Clusters"] == cl].iloc[:, 0].index)
    print("-" * 100, "\n")
```

```
In [ ]: df_currencies_t[df_currencies_t["HC_Clusters"] == 0].iloc[:, 0].index
```

Cluster one has the Kenya shilling, Argentine peso, Colombian peso, Nigerian naira, Russian ruble, Chilean peso, Indonesian rupiah, Indian rupee, Turkish lira, Mexican peso, and South African rand.

Cluster two has the Israeli shekel, Chinese yuan, Singaporean dollar, Taiwanese dollar, British pound, Australian dollar, New Zealand dollar, Canadian dollar, Swiss franc, Norwegian krone, and Swedish krona.

Cluster three has the Philippine peso, Malaysian ringgit, Euro, Hungarian forint, Romanian leu, South Korean won, Thai bhat, Poland zloty and Japanese yen.

We can see from the clusters that most developing countries are in the first cluster while developed nations are mostly in the second cluster.

We can also draw the dendrogram using the methodology below.

```

In [ ]: hier_average = linkage(subset_scaled_df.T, method="average", metric="euclidean")

In [ ]: hier_ward = linkage(subset_scaled_df.T, method="ward", metric="euclidean")

In [ ]: hier_comp = linkage(subset_scaled_df.T, method="complete", metric="euclidean")

In [ ]: # Change the chart style...
plt.style.use("fivethirtyeight")

In [ ]: df_currencies_t = df_currencies.T

plt.figure(figsize=(10, 10))
plt.title(
    "Dendrogram of FX Clusters, Jan 2019 through Jun 2022 (Complete)", fontsize=
)
plt.xlabel("Distance", fontsize=10)
plt.ylabel("Currency", fontsize=10)
plt.suptitle(
    "Fig. 12: Dendrogram with Complete Linkage",
    fontweight="bold",
    horizontalalignment="right",
)
dendrogram(
    hier_comp,
    orientation="right",
    # leaf_rotation=90.,
    leaf_font_size=20,
    labels=df_currencies_t.index.values,
    color_threshold=3,
)
plt.yticks(fontsize=6)
plt.show()

```

5. Conclusion

In this lesson, we have studied and implemented hierarchical clustering. In the next lesson, we will focus on another unsupervised learning method that is used to reduce the number of variables in a dataset.

References

1. Murtagh, Fionn, and Pedro Contreras. "Algorithms for Hierarchical Clustering: An Overview." *Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery*, vol. 2, no. 1, 2012.
2. Murtagh, Fionn, and Pedro Contreras. "Algorithms for Hierarchical Clustering: An Overview, II." *Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery*, vol. 7, no. 6, 2017.
3. Murtagh, Fionn, and Pedro Contreras. "Methods of Hierarchical Clustering." *arXiv preprint*, arXiv:1105.0121, 2011.
4. Nielsen, Frank. "Hierarchical Clustering." *Introduction to HPC with MPI for Data Science*. Springer, 2016, pp. 195-211.

MACHINE LEARNING IN FINANCE

MODULE 2 | LESSON 3

DIMENSIONALITY REDUCTION: PRINCIPAL COMPONENT ANALYSIS

Reading Time	30 minutes
Prior Knowledge	Unsupervised learning, correlation, eigenspaces
Keywords	principal components, loadings

In the first lesson, we introduced the concept of unsupervised learning, studied clustering algorithms, and examined how they identify structure in a given dataset. In the previous lesson, we applied these concepts using hierarchical clustering. In this lesson, we shift our focus to another topic in unsupervised learning: dimensionality reduction

1. Introduction

Dimensionality reduction algorithms compress data with many features into one with fewer features, while still retaining the most important information and eliminating information we consider redundant or duplicating. This is done by projecting a high dimensional dataset to a lower dimensional space.

A common misconception in machine learning is that the more features we add to an algorithm, the more accurate the model becomes. This has been disproven. It has been shown that to improve the performance of machine learning algorithms we should consider applying dimensionality reduction to our features so as to only remain with important and optimal features for the machine learning task of interest. Dimensionality reduction helps prevent the *curse of dimensionality* problem, that is to say, machine learning algorithms become inefficient in training data with bigger feature spaces.

The most common techniques in dimensionality reduction are

1. Feature selection
2. Feature extraction

Dimensionality reduction finds application in

1. Feature engineering
2. Noise reduction or filtering techniques
3. Generating artificial data for machine learning tasks.

4. Financial modeling or risk analysis by combining a huge number of market metrics to fewer components that allow for efficient modeling

Let's discuss the curse of dimensionality in detail.

1.1 Curse of Dimensionality

Machine learning models will often underfit on data with few features and overfit on data with too many features. This is what we refer to as the curse of dimensionality. It blocks many machine learning algorithms from achieving reasonable results as they make optimization of algorithms infeasible.

As dimension of data increases, difficulty in recognizing patterns also increases.

1.2 Dimensionality Reduction

Reducing dimensions of a dataset removes redundant features from the dataset but retains optimal variables that can be applied to predict target variables more accurately.

Dimensionality reduction also reduces the complexity of the model. A critical step in dimensionality reduction is the covariance or correlation matrix, and it helps check the correlation between our variables.

1.3 Correlation

Variables that are highly correlated with one another result in the variables' coefficients being unstable and, when used in regression, perform poorly. In machine learning, we use multicollinearity to find existence of correlation between independent variables.

The presence of multicollinearity is checked using Variance Inflation Factor (VIF) given by
$$\text{VIF} = \frac{1}{1 - R^2}$$

where $R^2 = 1 - \frac{SS_{\text{RES}}}{SS_{\text{TOT}}} = 1 - \frac{\sum_i (y_i - \hat{y})^2}{\sum_i (y_i - \bar{y})^2}$.

The VIF will always be greater than or equal to 1 and is interpreted as follows:

1. A VIF equal to one implies that the variable is independent of the other variables.
2. A VIF between 1 and 5 means the variables are moderately correlated.
3. A VIF greater than 5 implies that the variables are highly correlated.

Therefore, for a desirable predictive model, we will want the VIF to be as close to one as possible.

Correlation values range from -1 to +1 and the values are interpreted below:

1. Values close to +1 show a positive correlation.
2. Values close to -1 show a negative correlation.
3. Values close to 0 mean there exists no correlation between the two variables.

We can present the correlation between variables in a heatmap plot as shown below:

```
In [ ]: import matplotlib.pyplot as plt
import numpy as np
from sklearn import datasets

In [ ]: # Load Iris Dataset
iris = datasets.load_iris()

# Create a Covariance Matrix
cov_data = np.corrcoef(iris.data.T)
cov_data

In [ ]: # Plotting the covariance matrix using a heatmap

img = plt.matshow(cov_data, cmap=plt.cm.rainbow)
plt.colorbar(img, ticks=[-1, 0, 1], fraction=0.045)
for x in range(cov_data.shape[0]):
    for y in range(cov_data.shape[1]):
        plt.text(
            x,
            y,
            "%0.2f" % cov_data[x, y],
            size=12,
            color="black",
            ha="center",
            va="center",
        )

plt.show()
```

From the heatmap above we see that the first and third features, first and fourth features, and third and fourth features are correlated while the second feature can be considered independent from the other features.

Therefore, of the three highly correlated features, we can keep one and drop the other two, which in this case we consider duplication of information.

As mentioned earlier, we can use the following techniques for dimensionality reduction.

- Feature selection
- Feature extraction

Feature Selection

In this technique, we select a subset of the original variables, that is, choose variables that better represent other variables.

We can employ feature selection in one of the following instances:

- When the features are non-numeric
- When we need to extract important features
- To retain the measurements as they were originally without transformation.

We can observe feature selection using the following vector equation $\vec{x} = \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ \vdots \\ x_N \end{pmatrix} \rightarrow \begin{pmatrix} x_{i1} \\ x_{i2} \\ \vdots \\ x_{ik} \end{pmatrix}$ where $k < N$

Feature selection, however, requires a search strategy to choose the best features, and this may pose a challenge to an unsupervised learning model. Absence of class information makes it difficult to measure which features will improve the performance of a machine learning model. Common feature selection techniques without class information are done by selecting variables with higher variances.

Feature Extraction

In this method, we transform raw features to new features by mapping from the original features $\vec{x} = \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ \vdots \\ x_N \end{pmatrix} \rightarrow f(\vec{x}) = \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_k \end{pmatrix}$ where $k < N$.

The transformation can be linear or non-linear.

The linear feature extraction techniques are listed below:

1. Principal Component Analysis (PCA)
2. Linear Discriminant Analysis (LDA)

In this lesson, we will focus on PCA.

2. Principal Component Analysis (PCA)

PCA is a useful technique to reduce dimensionality of our feature set while retaining information from the original features. It does this through feature extraction, that is, new features are formed by performing a linear combination of the original features. By applying PCA to a dataset, we are able to learn different representations of the data explaining the original data points.

The linear transformation of the features can be represented by the equation $y = WX$ where W are the weights, X the original features and y , the principal components.

The linear transform poses challenge in that it makes the transformed features less interpretable and therefore not useful for follow-up analysis.

The principal components, y_i , explains a higher percent of the original data's variability and the weights W_{ij} show the contribution of our original features to the resulting principal components.

Consider an example with two variables; $y_i = w_{i,1} x_1 + w_{i,2} x_2$

We refer to the weights $(w_{i,1}, w_{i,2})$ as the component **loadings**. The first principal component, y_1 , explains most of the variation in our dataset, y_2 which is orthogonal to y_1 explains most of the remaining variations. If we had more features, then the additional principal components would be orthogonal to each other as well.

The new variables we find can be linearly combined to get back our original data features. These new variables are referred to as the principal components (PCs) and the PCs are orthogonal (independent) of each other.

Principal component analysis can be used in the following ways:

1. Visualizing a high dimensional dataset into lower dimensional plots.
2. Removing noise from a data.
3. Finding structure in a dataset.
4. Reduce overfitting in a data.

Mathematically we can generalize the objectives of PCA as:

- To find an orthonormal basis for a dataset.
- Arrange dimensions sequentially in the order of their importance.
- Remove dimensions with lower significance.
- Ensure the new variables are uncorrelated.

2.1 Computing the Principal Components and Loadings

Calculating the principal components involves the use of a correlation matrix of the data or a covariance matrix, and it only works on numerical features. These are the steps to performing dimensionality reduction.

1. Standardization: The first step in a PCA process is to standardize the dataset. This is necessary because PCA is sensitive to datasets whose features have different value ranges or have outliers. This step would therefore ensure that each feature contributes equally. Standardization is achieved by subtracting from a feature its mean and dividing by its standard deviation. The standardized feature would therefore have a mean of zero and standard deviation of one (unit length of one).
2. Centering data around the origin: We proceed by calculating the average of each feature and find the center of the data. We shift the data so that our center is at the origin without altering the distance between the data points.
3. The first principal component (PC1) is computed by finding a linear combination of the features that maximizes the total variation in the data $Y_1 = a_{11} X_1 + a_{12} X_2 + \dots + a_{1N} X_N$. The implication of this is that we can increase the variance of Y_1 to be as large as possible. To prevent such an occurrence, the sum of squares of the weights is equal to 1, that is, $a_{11}^2 + a_{12}^2 + \dots + a_{1N}^2 = 1$.
4. We repeat this process by finding the principal component (PC2) that passes through the origin and is orthogonal (i.e., uncorrelated) to PC1 and accounts for the next high variance. $Y_2 = a_{21} X_1 + a_{22} X_2 + \dots + a_{2N} X_N$
5. This process continues until we have as many principal components as the original features. Once we have all the principal components, then the total variance of the

principal components will equal the total variance of the original features, and with this, we are certain that no information has been lost in the transformation process. As seen in the steps above, the PCA rotates data samples in the direction of the axes, then shifts the data so that their mean equals 0 but most importantly ensuring that no information is lost in the process.

In matrix form, finding PCA can be written as $Y = XA$

The rows of the matrix A are the eigenvectors and they specify how the principal components are oriented compared to the original features. The composition of the eigenvectors are the weights a_{ij} and are referred to as the loadings. Loadings tell us how each feature contributes to a principal component.

Loadings range from -1 to 1 and large loadings (close to 1 or -1) indicate the presence of a strong relationship between a particular feature and the principal component while loadings closer to zero indicate a weaker relationship. The sign of the loading shows whether a feature is positively or negatively correlated to the principal component.

Given a matrix A and the variance-covariance matrix of our original data, S_X , then the variance-covariance matrix of the resulting principal components will be $S_Y = A S_X A^T$

S_Y is a diagonal matrix whose values are the eigenvalues (or the explained variance) by the corresponding principal components. The PCA performs dimensional reduction by projecting data onto the principal component space and then orders the sequence of PCs in the direction of the largest variance in our dataset. This ensures that the PCs are not correlated with each other and they form a set of orthogonal basis. Therefore the eigenvalues are arranged such that they decrease monotonically from the largest to the smallest. We use **scree plot** to visualize the rate at which the explained variance is decreasing as we add more principal components.

An ideal scree plot would steadily bend and resemble an elbow plot, where insignificant PCs will flatten out and we get a nice cut-off allowing us to pick the number of optimal PCs easily. In case the scree plot is not so ideal, we can use the rules below to choose the PCs.

1. Apply the Kaiser rule, that is, the PCs should have an eigenvalue of at least 1.
2. The selected PCs should contribute at least 80% of the variance.

In case we get more than 3 PCs, we should consider using other dimensional reduction techniques as the PCA would not be the best way to visualize our data. In the new coordinate system, the position of the principal components are called the scores.

Below is a recap of the eigenspaces that we have learned above.

2.2 Mathematics behind PCA

Below we review the concept of eigenspaces;

Let $\mathbf{A}$ be an $n \times n$ matrix. The eigenvalues of $\mathbf{A}$ are the solution to the characteristic equation $\det(\mathbf{A} - \lambda \mathbf{I}) = 0$ where $\mathbf{I}$ is an $n \times n$ identity matrix.

If λ is an eigenvalue of $\mathbf{A}$, then there exists a vector $\mathbf{x}$ such that $\mathbf{A} \mathbf{x} = \lambda \mathbf{x}$

This vector $\mathbf{x}$ is referred to as the eigenvector of the matrix $\mathbf{A}$ associated with the eigenvalue λ and it is not unique.

Consider a 2×2 matrix $\mathbf{A}$ with eigenvectors $\mathbf{x}_1, \mathbf{x}_2$ and eigenvalues λ_1, λ_2 . From the definition of eigenvectors, then $\mathbf{A} \mathbf{x}_1 = \lambda_1 \mathbf{x}_1$ and $\mathbf{A} \mathbf{x}_2 = \lambda_2 \mathbf{x}_2$ which can be written as: $\mathbf{A} \begin{bmatrix} \mathbf{x}_1 & \mathbf{x}_2 \end{bmatrix} = \begin{bmatrix} \mathbf{x}_1 & \mathbf{x}_2 \end{bmatrix} \begin{bmatrix} \lambda_1 & 0 \\ 0 & \lambda_2 \end{bmatrix}$.

Let $\Phi = \begin{bmatrix} \mathbf{x}_1 & \mathbf{x}_2 \end{bmatrix}$ and $\Lambda = \begin{bmatrix} \lambda_1 & 0 \\ 0 & \lambda_2 \end{bmatrix}$. This translates to: $\mathbf{A} \Phi = \Phi \Lambda$

Normalizing the eigenvectors such that they are orthogonal we have; $\Phi^T \Phi = \mathbf{I}$ and $\Phi \Phi^T = \mathbf{I}$ which implies $\Phi^T \mathbf{A} \Phi = \Lambda$ and $\mathbf{A} = \Phi \Lambda \Phi^T$.

How does this apply in the PCA process?

Let S be an $m \times m$ covariance (correlation) matrix, there exists an orthogonal $m \times m$ matrix, Φ , such that the eigenvectors of S are the columns of Φ and the eigenvalues of S are the diagonal of the matrix Λ such that: $\Phi^T S \Phi = \Lambda$. The eigenvectors of the covariance (correlation) matrix give us the direction vectors Φ_1, Φ_2 . We can create a transformation matrix from the eigenvectors that transform our data from the $[x_1, x_2]$ axis system to the $[\Phi_1, \Phi_2]$ axis system using equation $p_{\Phi} = (p_x - \mu_x) \cdot \Phi$ with p_x being any point in the $[x_1, x_2]$ axis system and μ_x is the data mean.

2.3 Implementation of PCA

We start by installing the PCA library.

Then, import libraries we need for this **section**.

```
In [ ]: import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from pca import pca
```

The next step is to create a dataset on which we will apply our PCA algorithm.

```
In [ ]: # Dataset
from sklearn.datasets import load_wine

# Load dataset
```

```
data = load_wine()
X = data.data
y = data.target
labels = data.feature_names
# Make dataframe
df = pd.DataFrame(data=X, columns=labels, index=y)
```

```
In [ ]: X.shape
```

```
In [ ]: # Initialize with normalization
model = pca(normalize=True, n_components=None)
```

```
In [ ]: # Fit transform with dataframe
out = model.fit_transform(df)
```

The scree plot below helps us see the percentage of variance captured by all the PCs. We may need to look at other feature reduction techniques as it takes 5 variables to get a cumulative percentage variance of 80%.

```
In [ ]: # Scree plot with explained variance
print("\nFig. 1: Scree Plot" + "\n")
model.plot()
```

We can now see the loadings in the code below. The loadings as discussed in the previous sections assist us in seeing how the features contribute to the PCs and how classes are divided.

```
In [ ]: # Top performing features
model.results["topfeat"]
```

We can now visualize the scatter plot of the scores as shown below.

```
In [ ]: # Make plots

# Scatter plot
print("\nFig. 2: Scatter plot of the Scores" + "\n")
fig, ax = model.scatter(figsize=(10, 6))
```

From the scatter plot, we draw the biplot and note that the first 2 PCs can be sufficiently used to separate the first 3 classes.

```
In [ ]: # Biplot
print("\nFig. 3: Biplot" + "\n")
fig, ax = model.biplot(figsize=(10, 6))
```

For a clear picture of the biplot without the scores, see below. Note that the lesser the angle between two features indicates that they contribute heavily to a principal component.

```
In [ ]: print("\nFig. 3: Biplot without the scores" + "\n")
fig, ax = model.biplot(cmap=None, label=False, legend=False, figsize=(10, 6))
```

3. Conclusion

In this lesson, we introduced the concept of Principal Component Analysis, and in the next lesson, we will apply this knowledge to yield data.

References

- Abdi, Hervé, and Lynne J. Williams. "Principal Component Analysis." *Wiley Interdisciplinary Reviews: Computational Statistics*, vol. 2. no. 4, 2010, pp. 433-459.
- Ringnér, Markus. "What is Principal Component Analysis?" *Nature Biotechnology*, vol. 26, no. 3, 2008, pp. 303-304.

Copyright 2025 WorldQuant University. This content is licensed solely for personal use. Redistribution or publication of this material is strictly prohibited.

MACHINE LEARNING IN FINANCE

MODULE 2 | LESSON 4

PRINCIPAL COMPONENT ANALYSIS AND INTEREST RATE MODELING

Reading Time	30 Minutes
Prior Knowledge	PCA, Machine learning
Keywords	Yield curves, covariance, VaR, Loadings

In the previous lesson, we studied principal component analysis (PCA) and the mathematical theory behind it. In this lesson, we will implement PCA on treasury rates and use the resultant principal components to calculate the Value at Risk.

1. Introduction

1.1 Yield Curve

A yield curve is a line plot of bond interest rates of the same credit quality, having different maturities. The yield curve has two dimensions to it:

- It depicts investors' average market expectations on the performance of future short-term bonds.
- The term premium - When the yield curve is upward sloping, it shows investors' confidence in the market as they are optimistic about receiving better returns in long-term bonds. In instances where the curve is downward sloping, investors expect tough times like a recession ahead. The yield curve could also be flat depicting no change in short-term and long-term rates.

These features of the yield curve can be modeled using principal components.

1.2 Value at Risk (VaR)

In this subsection, we only cover a small portion of VaR that will be useful to a later topic.

VaR is a useful metric to measure market risk on portfolio performance. VaR can be thought of as the largest possible loss we can incur for a given confidence interval and time period. VaR is a numerical value that distinguishes the tail (loss) of a distribution from the rest of the distribution.

An individual position-level VaR is calculated by multiplying the position volatility (standard deviation) of return, the portfolio total value, the absolute of proportion and the z -score of our chosen confidence interval.

z -score is used since we assume that our returns follow a normal distribution. For example, the z -score of a one-tailed 99% VaR can be read in the [normal table](#) as equal to 2.32635.

In this lesson we will focus on portfolio-level VaR, which is calculated as:
$$\text{Portfolio Var} = \text{Portfolio Volatility} \times \text{Portfolio Total Value} \times \text{z-score of confidence interval}$$
 where the portfolio volatility is the standard deviation of the returns in our portfolio.

2. Principal Component Analysis

Principal component analysis can be used to handle risk arising from data where variables are highly correlated. In this approach, we consider the historical data describing the market variables' movement and define factors that will explain the movements.

We will consider yield data with maturities of 1 month, 3 months, 6 months, 1 year, 2 years, 3 years, 5 years, 7 years, 10 years, 20 years, and 30 years.

2.1 Loading Packages

We start by loading helper packages that will help us achieve our tasks.

```
In [ ]: # Load Libraries
# Global Libraries
# Disable the warnings
import warnings
from datetime import datetime

import numpy as np
import pandas as pd
import pandas_datareader.data as web
import scipy as sp

# Plotting
import seaborn as sns
from matplotlib import pyplot
from sklearn.preprocessing import StandardScaler

warnings.filterwarnings("ignore")

from sklearn.decomposition import PCA
```

2.2 Loading Data

We download U.S. yield data ranging from 1-month to 30-year rates.

```
In [ ]: # downloading the data
start = datetime(2002, 6, 30)
end = datetime(2022, 6, 30)
data = [
    "DGS1M0",
    "DGS3M0",
    "DGS6M0",
    "DGS1",
    "DGS2",
    "DGS3",
    "DGS5",
    "DGS7",
    "DGS10",
    "DGS20",
    "DGS30",
]
data = web.DataReader(data, "fred", start, end).dropna(how="all").ffill()
```

```
In [ ]: data.rename(
    columns={
        "DGS1M0": "1m",
        "DGS3M0": "3m",
        "DGS6M0": "6m",
        "DGS1": "1y",
        "DGS2": "2y",
        "DGS3": "3y",
        "DGS5": "5y",
        "DGS7": "7y",
        "DGS10": "10y",
        "DGS20": "20y",
        "DGS30": "30y",
    },
    inplace=True,
)
df = data.copy()
df.head(10)
```

2.3 Exploratory Data Analysis

```
In [ ]: df.shape
```

```
In [ ]: df2 = df.copy()
```

Now let's visualize the movement of the yield curves.

```
In [ ]: df.plot(figsize=(15, 8))
pyplot.ylabel("Rate")
pyplot.legend(bbox_to_anchor=(1.01, 0.9), loc=2)
pyplot.suptitle(
    "Fig. 1: Yield Curve Movement", fontweight="bold", horizontalalignment="right"
)
pyplot.show()
```

3. Generating Principal Components

In this section, we study how to generate principal components from a given dataset.

3.1 Using the Covariance Matrix

The day-to-day data from financial markets usually have two properties

- Noise
- Signal

When we apply PCA to the dataset, we can extract its signal and find the minimum quantity of data that will account for the largest percentage of the whole dataset.

We then detrend the data by standardizing it into z-scores.

```
In [ ]: df_std = (df - df.mean()) / df.std()  
df_std.head()
```

Let us derive the covariance matrix of the standardized data above.

```
In [ ]: cov_matrix_array = np.array(np.cov(df_std, rowvar=False))  
cov_matrix_array
```

```
In [ ]: cov_df = pd.DataFrame(cov_matrix_array, columns=df.columns, index=df.columns)  
cov_df
```

Then, perform eigendecomposition on the covariance matrix and find the percentage of the eigenvectors as a percentage of the total variance.

```
In [ ]: # Perform eigendecomposition  
  
eigenvalues, eigenvectors = np.linalg.eig(cov_matrix_array)  
  
# Put data into a DataFrame and save to excel  
df_eigval = pd.DataFrame({"Eigenvalues": eigenvalues})  
  
eigenvalues
```

```
In [ ]: # We calculate explained variance  
  
explained_variance = [round(variance / sum(eigenvalues), 3) for variance in eigenvalues]  
explained_variance
```

```
In [ ]: # Save output to Excel  
columns = [  
    "PC1",  
    "PC2",  
    "PC3",  
    "PC4",  
    "PC5",  
    "PC6",  
    "PC7",  
    "PC8",  
    "PC9",  
    "PC10",  
    "PC11",  
]
```

```

]
df_eigvec = pd.DataFrame(eigenvectors, columns=columns, index=df.columns)

df_eigvec.to_excel("df_eigvec.xlsx")
eigenvectors[0]

```

```
In [ ]: df_eigvec
```

The indices in the table above are the maturities of the rates that we considered for this exercise while the columns in our dataframe are the factor loadings describing the rate movements.

Working out the combined variation of the components:

```
In [ ]: from itertools import accumulate

df_eigval["Explained proportion"] = df_eigval["Eigenvalues"] / np.sum(
    df_eigval["Eigenvalues"]
)
df_eigval["Cumulative Explained Variance"] = list(
    accumulate(df_eigval["Explained proportion"])
)

# Format as percentage
df_eigval.style.format({"Explained proportion": "{:.2%}"})
df_eigval.style.format({"Cumulative Explained Variance": "{:.2%}"})

```

```
In [ ]: fig, ax = pyplot.subplots()

pyplot.plot(df_eigvec["PC1"])
pyplot.suptitle(
    "Fig. 2: The First Factor Loading", fontweight="bold", horizontalalignment="
)
pyplot.ylabel("Factor Loadings")
ax.set_xticks(np.arange(11))
ax.set_xticklabels(df.columns, rotation="vertical")
pyplot.xlabel("Term")
pyplot.show()

```

The first factor loading (PC1) is the weighted combination of all the rates and represents a parallel shift in our yield curve. We can see that for one unit of the loading, the 1-month rate increases by \$0.299\$ basis points, the 2-month rate increases by \$0.302\$ basis points, and so on.

```
In [ ]: fig, ax = pyplot.subplots()

pyplot.plot(df_eigvec["PC2"])
pyplot.suptitle(
    "Fig. 3: The Second Factor Loading", fontweight="bold", horizontalalignment="
)
pyplot.ylabel("Factor Loadings")
ax.set_xticks(np.arange(11))
ax.set_xticklabels(df.columns, rotation="vertical")
pyplot.xlabel("Term")
pyplot.show()

```

The second factor loading (PC2) represents a change of slope of the yield curve when the short end moves in the opposite direction of the long end zone. We can observe that the rates for 1 month to 4 years move in one direction and the rates between 5 years and 30 years move in the other direction.

```
In [ ]: fig, ax = pyplot.subplots()

pyplot.plot(df_eigvec["PC3"])
pyplot.suptitle(
    "Fig. 4: The Third Factor Loading", fontweight="bold", horizontalalignment="
)
pyplot.ylabel("Factor Loadings")

ax.set_xticklabels(df.columns, rotation="vertical")
pyplot.xlabel("Term")
pyplot.show()
```

The third factor loading (PC3) represents a twist of the sovereign yield curve, which happens when the short- and long-end segments move up simultaneously as the yield moves down.

We can now plot the scree plot to visualize the factor loadings' contribution to the variance of the dataset. /+

```
In [ ]: PC_values = np.arange(11)
pyplot.plot(
    PC_values, df_eigval["Explained proportion"] * 100, "o-", linewidth=2, color
)
pyplot.plot(
    PC_values,
    df_eigval["Cumulative Explained Variance"] * 100,
    "o-",
    linewidth=2,
    color="red",
)
pyplot.suptitle("Fig. 5: Scree Plot", fontweight="bold", horizontalalignment="ri
pyplot.title("Scree Plot")
pyplot.xlabel("Principal Component")
pyplot.ylabel("Variance Explained")
pyplot.legend(["Individual Variance", "Cumulative Variance"])
pyplot.show()
```

We can see that the first two components describe more than 99% of the variance and can therefore be used to describe the yield curve movement.

Now let's calculate the principal components, which are the dot product of the standardized data and the eigenvectors.

```
In [ ]: principal_components = df_std.dot(eigenvectors)
principal_components.columns = df_eigvec.columns
principal_components.head()
```

We now plot the first three components, which account for more than 99% of the explained variance in the data.

```
In [ ]: pyplot.plot(principal_components[["PC1", "PC2", "PC3"]])
pyplot.xlabel("Time (Years)")
pyplot.suptitle(
    "Fig. 6: Principal Component", fontweight="bold", horizontalalignment="right"
)
pyplot.legend(["PC1", "PC2", "PC3"], bbox_to_anchor=(1.01, 0.9), loc=2)
pyplot.show()
```

The principal components do not tell us much, but from the diagram above, we can see that the first principal component is more volatile than the other two, which is expected.

Below is a function that sums up the PCA process we have seen above.

```
In [ ]: def PCA_CALC(df, num_reconstruct):
    df -= df.mean(axis=0) / df.std()
    R = np.cov(df, rowvar=False)
    eigen_values, eigen_vectors = sp.linalg.eigh(R)
    eigen_vectors = eigen_vectors[:, np.argsort(eigen_values)[::-1]]
    eigen_values = eigen_values[np.argsort(eigen_values)[::-1]]
    eigen_vectors = eigen_vectors[:, :num_reconstruct]

    return np.dot(eigen_vectors.T, df.T).T, eigen_values, eigen_vectors
```

```
In [ ]: scores, evals, evecs = PCA_CALC(df, 11)
```

```
In [ ]: fig, ax = pyplot.subplots()

evecs = pd.DataFrame(evecs)
pyplot.plot(evecs.loc[:, 0:2])
pyplot.suptitle(
    "Fig. 7: The Factor Loadings", fontweight="bold", horizontalalignment="right"
)
pyplot.legend(["PC1", "PC2", "PC3"], loc="lower right")
pyplot.ylabel("Factor Loadings")
ax.set_xticks(np.arange(11))
ax.set_xticklabels(df.columns, rotation="vertical")
pyplot.xlabel("Term")
pyplot.show()
```

As discussed in the previous subsection, PC1 reflects a (mostly) parallel shift, PC2 reflects a tilt, and PC3 reflects a twist or butterfly.

We now work to reconstruct the original data from our factor loadings.

```
In [ ]: reconst = pd.DataFrame(np.dot(scores, evecs.T), index=df.index, columns=df.columns)

reconst.plot(figsize=(15, 8))
pyplot.ylabel("Rate")
pyplot.xlabel("Years")
pyplot.title("Reconstructed Mean-Subtracted Dataset")
pyplot.suptitle(
    "Fig. 8: Reconstructed Mean-SUBtracted Dataset",
    fontweight="bold",
```

```

        horizontalalignment="right",
    )
    pyplot.legend(bbox_to_anchor=(1.01, 0.9), loc=2)
    pyplot.show()

```

We now reconstruct our original dataset from the principal components and plot the outcome.

```

In [ ]: for cols in reconst.columns:
        reconst[cols] = reconst[cols] + df2.mean(axis=0)[cols]

reconst.plot(figsize=(15, 8))
pyplot.xlabel("Date")
pyplot.ylabel("Rates")
pyplot.title("Reconstructed Initial Dataset")
pyplot.suptitle(
    "Fig. 9: Reconstructed Initial Dataset",
    fontweight="bold",
    horizontalalignment="right",
)
pyplot.legend(bbox_to_anchor=(1.01, 0.9), loc=2)
pyplot.show()

```

Note that these steps can be implemented using a scikit learn module as shown below.

We start by computing the correlation between the term interests.

```

In [ ]: # correlation
correlation = df.corr()
pyplot.figure(figsize=(15, 15))
pyplot.title("Correlation Matrix")
pyplot.suptitle(
    "Fig. 10: Correlation Matrix", fontweight="bold", horizontalalignment="right"
)
sns.heatmap(correlation, vmax=1, square=True, annot=True, cmap="cubehelix");

```

Then, standardize the dataset as explained earlier. Note that when the variable scales are different, we use a correlation matrix, and when the variable scales are similar, we use a covariance matrix.

```

In [ ]: scaler = StandardScaler().fit(df)
rescaledddf = pd.DataFrame(scaler.fit_transform(df), columns=df.columns, index=df)
# summarize transformed data
df.dropna(how="any", inplace=True)
rescaledddf.dropna(how="any", inplace=True)
rescaledddf.head(2)

```

See the visualization of the scaled dataset below.

```

In [ ]: rescaledddf.plot(figsize=(14, 10))
pyplot.ylabel("Rate")
pyplot.legend(bbox_to_anchor=(1.01, 0.9), loc=2)
pyplot.suptitle(
    "Fig. 11: Scaled Yield Curve Plot", fontweight="bold", horizontalalignment="right"
)
pyplot.show()

```

We call the PCA algorithm and fit our data to it, then data transformation takes place here.

```
In [ ]: model = PCA()
PrincipalComponent = model.fit(rescaledddf)
```

We now visualize the explained variance of our components.

```
In [ ]: NumEigenvalues = 5
fig, axes = pyplot.subplots(ncols=2, figsize=(14, 4))
pd.Series(model.explained_variance_ratio_[:NumEigenvalues]).sort_values(
    ascending=False
).plot.bar(title="Explained Variance Ratio by Top Factors", ax=axes[0])
pd.Series(model.explained_variance_ratio_[:NumEigenvalues]).cumsum().plot(
    ylim=(0, 1), ax=axes[1], title="Cumulative Explained Variance"
)

# explained_variance
pd.Series(np.cumsum(model.explained_variance_ratio_)).to_frame(
    "Explained Variance_Top 5"
).head(NumEigenvalues).style.format("{:,.2%}".format)
pyplot.suptitle("Fig. 12: Scree Plots", fontweight="bold", horizontalalignment="right")
pyplot.show()
```

```
In [ ]: pyplot.plot(model.components_[0:3].T)
pyplot.xlabel("Principal Component")
pyplot.suptitle(
    "Fig. 13: Factor Loadings", fontweight="bold", horizontalalignment="right"
)
pyplot.legend(["PC1", "PC2", "PC3"], loc="lower right")
pyplot.show()
```

Again, let's not forget that PC1 reflects a (mostly) parallel shift, PC2 reflects a tilt, and PC3 reflects a twist or butterfly.

Let's create a dataframe having only the first 3 components, which is what we are interested in.

```
In [ ]: model = PCA().fit(rescaledddf)
columns = ["pca_comp_%i" % i for i in range(11)]
df_pca = pd.DataFrame(
    model.transform(rescaledddf), columns=columns, index=rescaledddf.index
)
df_pca.head()
```

```
In [ ]: # get the component variance
# Proportion of Variance (from PC1 to PC11)
model.explained_variance_ratio_
```

```
In [ ]: # Cumulative proportion of variance (from PC1 to PC6)
np.cumsum(model.explained_variance_ratio_)
```

```
In [ ]: # component loadings or weights (correlation coefficient between original variab
# component loadings represents the elements of the eigenvector
# the squared loadings within the PCs always sums to 1
```

```

loadings = model.components_
num_pc = model.n_features_
pc_list = ["PC" + str(i) for i in list(range(1, num_pc + 1))]
loadings_df = pd.DataFrame.from_dict(dict(zip(pc_list, loadings)))
loadings_df["variable"] = df.columns.values
loadings_df = loadings_df.set_index("variable")
loadings_df

```

```

In [ ]: # positive and negative values in component loadings reflects the positive and n
# correlation of the variables with the PCs. Except A and B, all other variables
# positive projection on first PC.

# get correlation matrix plot for loadings

import seaborn as sns

sns.set(rc={"figure.figsize": (11.7, 8.27)})
import matplotlib.pyplot as plt

ax = sns.heatmap(loadings_df, annot=True, cmap="Spectral")
plt.suptitle(
    "Fig. 14: Correlation matrix for Loadings",
    fontweight="bold",
    horizontalalignment="right",
)
plt.show()

```

```

In [ ]: # get eigenvalues (variance explained by each PC)
model.explained_variance_

```

So far, the three approaches we have applied have yielded similar results. So we want to plot the biplot of the first two principal components.

```

In [ ]: from pca import pca

```

```

In [ ]: # Or reduce the data towards 2 PCs
model = pca(n_components=2)

# Fit transform
results = model.fit_transform(rescaled_df)

```

```

In [ ]: # Make biplot with the number of features
print("\033[1m" + "Fig. 15: Biplot without the scores" + "\033[0m")
fig, ax = model.biplot(cmap=None, label=False, legend=False, figsize=(10, 6))

```

From the biplot, we can see short-term interest rates are highly correlated, and this can also be said of the long-term interest rates, this is because the angle between them is smaller. Comparing short-term and long-term interest rates, we can conclude that they are negatively correlated as the angle between them is wider. The mid-term interest rates seem not to be correlated with both the short- and long-term interest rates.

4. Computing Value At Risk (VaR) using PCA

In this section, we will demonstrate how we can use PCA to calculate VaR. For the purposes of this lesson, let us assume we have a portfolio with the exposures to interest rate variation as shown in the table below:

```
In [ ]: df_portfolio = pd.DataFrame(
    {
        "2 year rate": [10],
        "3 year rate": [4],
        "5 year rate": [-8],
        "7 year rate": [-7],
        "10 year rate": [2],
    }
)

df_portfolio
```

The table reads as follows: A \$1\$ basis point would see our portfolio value increase by \$10\$ million considering the 2-year rate; similarly, we will see a \$4\$ million portfolio increase in a 3-year rate, and so on.

From our scree plot, we saw that two-factor loadings were sufficient to model our yield rates as they account for more than 99% of the data variance. Recall our loadings dataframe.

```
In [ ]: df_eigvec
```

We can now calculate the exposure brought about by the first factor as

```
In [ ]: 10 * 0.3169 + 4 * 0.322 - 8 * 0.3231 - 7 * 0.3143 + 2 * 0.2987
```

The exposure due to the second factor will be

```
In [ ]: 10 * 0.1856 + 4 * 0.0994 + 8 * 0.0798 + 7 * 0.2122 - 2 * 0.3285
```

The first factor has very little exposure as compared to the second factor.

The change in the portfolio value becomes $\Delta P = 0.2695 f_1 + 3.7204 f_2$

Recall the loadings variance below:

```
In [ ]: df_eigval
```

We can calculate the standard deviation (factor score) as shown below:

```
In [ ]: df_eigval["Factor_store"] = np.sqrt(df_eigval["Eigenvalues"])
df_eigval
```

The standard deviation of ΔP will therefore be

```
In [ ]: np.sqrt(np.square(0.2695) * 9.304 + np.square(3.7204) * 1.553)
```

Finally, the one day 99% VaR is given by $\sigma(\Delta P) \times Z_{\alpha}$

5. Conclusion

In this lesson, we have applied principal component analysis concept on yield rate data and seen how we can arrive at the same conclusion using three different approaches. We have used results to calculate VaR for our portfolio.

References

1. Carcano, Nicola. "Yield Curve Risk Management: Adjusting Principal Component Analysis for Model Errors." *The Journal of Risk*, vol. 12, no. 1, 2009, pp. 3–16.
2. Oprea, Andrea. "The Use of Principal Component Analysis (PCA) in Building Yield Curve Scenarios and Identifying Relative-Value Trading Opportunities on the Romanian Government Bond Market." *Journal of Risk and Financial Management*, vol. 15, no. 6, 2022. <https://www.mdpi.com/1911-8074/15/6/247/htm>.
3. Stat Trek. "Normal Distribution Calculator." <https://stattrek.com/online-calculator/normal>.